Secure Real-Time Voice Communication System using STM32 and LTE

Project Documentation & Implementation Plan

Team Members	College ID
Rawan Mohamed Ismail	221006089
Malak Wael Ghabn	221004011
Karim Mohamed Ismail	221004798
Omar Mohamed Sayed El-Shafei	221005236

Supervised By	Dr. Amr Fahmy
Course	Embedded Systems ECE5206
Semester	Semester 8
Department	Computer Engineering
Institution	Arab Academy for Science, Technology and Maritime Transport

1. Project Overview

1.1 Problem Statement

Existing voice communication systems either lack hardware-level encryption or are too computationally expensive for resource-constrained microcontrollers. This project addresses the need for a low-cost, embedded solution that delivers real-time encrypted voice communication over LTE, integrating ADC/DAC signal processing with RSA cryptography on an STM32 microcontroller.

1.2 System Goal

Build two identical communication nodes (Device A and Device B). Each node captures voice from a microphone, encrypts it on the STM32, transmits it over the 4G LTE network to the peer node, where it is decrypted on the peer's STM32 and played back through a speaker. Both nodes operate in full-duplex mode. Calls are initiated by phone number through a small relay server, which forwards the encrypted bytes between the two nodes.

1.3 Objectives

- Implement real-time voice capture, ADC conversion, and DAC reconstruction on STM32 with end-to-end latency under 200 ms (excluding network transit).
- Integrate RSA-512 + AES-128 hybrid encryption on the STM32 without external processing hardware.
- Establish bidirectional encrypted voice transmission over a 4G LTE network using two T-A7608SA-H modules.
- Achieve clear audio output with acceptable SNR using MAX9814 (capture) and LM386 (playback) amplification stages.

2. Bill of Materials (BOM)

Components for two complete, identical nodes plus shared programming hardware.

#	Component	Qty	Unit (LE)	Total (LE)
1	STM32F401RCT6 Black Pill development board	2	270.00	540.00
2	MAX9814 Electret Microphone Amplifier (AGC)	2	230.00	460.00
3	MCP4725 12-bit Digital-to-Analog Converter (I²C)	2	195.00	390.00
4	TRRS 3.5 mm Audio Jack Female AUX Breakout ¥3.0	4	5.00	20.00
5	ST-LINK V2 Emulator/Programmer	1	140.00	140.00
6	LM386 Audio Amplifier Module (200× gain)	2	35.00	70.00
7	T-A7608SA-H 4G LTE Module	2	(supplied)	—
	Total Estimated Cost			1,620.00 LE

2.1 Component Roles

STM32F401RCT6 Black Pill (×2) — Main microcontroller. Handles ADC capture, encryption, DAC playback, and UART communication with the LTE module. Runs at 84 MHz HCLK.

MAX9814 Microphone Amplifier (×2) — Electret microphone preamplifier with auto gain control. Outputs an AC-coupled audio signal biased at ~1.25 V into the STM32 ADC.

MCP4725 DAC (×2) — External 12-bit I²C DAC. Reconstructs the decrypted digital audio into an analog waveform at 8 kHz sample rate.

LM386 Audio Amplifier (×2) — Low-voltage power amplifier. Drives the headphone output from the DAC's low-current signal.

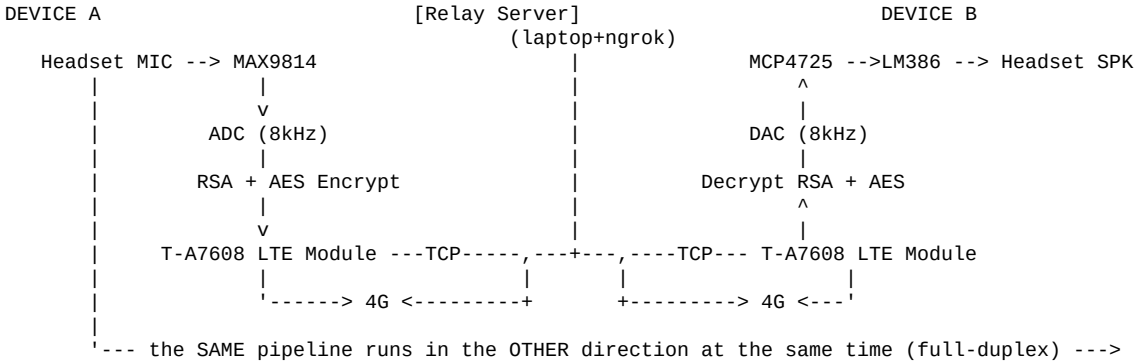
TRRS 3.5 mm Audio Jacks (×4) — Standard 4-conductor headphone jacks. Used to connect a TRRS headset (mic + speaker) to each node, providing one-cable audio I/O.

T-A7608SA-H 4G LTE Module (×2) — 4G LTE modem with TCP/IP stack. Driven over UART using AT commands. Carries encrypted voice packets between the two nodes via the relay server.

ST-LINK V2 (×1) — SWD programmer/debugger. Used during development to flash firmware to both Black Pill boards and to debug via breakpoints.

3. System Architecture

3.1 High-Level Block Diagram



Both devices run identical firmware. Each device simultaneously captures and encrypts outgoing audio while receiving and decrypting incoming audio, producing full-duplex (two-way) communication.

3.2 Why a Relay Server?

Both T-A7608 LTE modules connect to the cellular network through carrier-grade NAT (CGNAT). They are assigned private IP addresses and have **no public address**, so they cannot reach each other directly. A small relay server with a public IP receives connections from both nodes and forwards encrypted bytes between them.

Critical: the relay never sees plaintext audio. It only forwards ciphertext bytes. End-to-end encryption remains on the STM32 nodes, exactly as production voice apps (WhatsApp, Signal, FaceTime) operate.

3.3 Phone-Number-Based Calling Flow

1. Both nodes boot, bring up LTE data, and connect TCP to the relay's public address.
2. Each node sends **REG <phone>** to register its phone number with the relay.
3. User on Device A presses the Call button → A sends **CALL <B's phone>**.
4. Relay sends **INC <A's phone>** to Device B → B's status LED indicates ringing.
5. User on Device B presses Answer → B sends **ANS** → relay sends **GO** to both ends.
6. From this point, the TCP channel carries encrypted audio in both directions, transparently forwarded by the relay.
7. Either side can send **HUP** to terminate the call.

4. Hardware Configuration

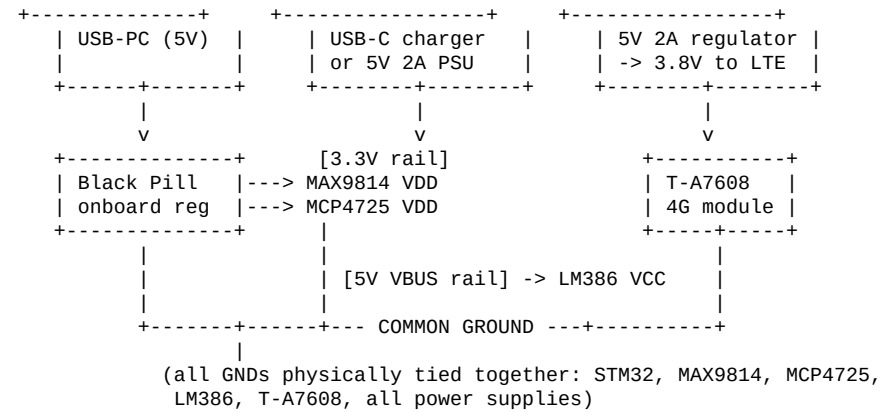
4.1 STM32F401RCT6 Pin Assignment

Final pin map per node. All other pins are unused and can be ignored.

Pin	Function	Connects to
PA0	ADC1_IN0	MAX9814 OUT
PA2	USART2_TX	USB-TTL adapter RX (optional debug log)
PA3	USART2_RX	USB-TTL adapter TX (optional debug log)
PA9	USART1_TX	T-A7608 LTE module RXD
PA10	USART1_RX	T-A7608 LTE module TXD
PB6	I2C1_SCL	MCP4725 SCL
PB7	I2C1_SDA	MCP4725 SDA
PA13	SWDIO	ST-LINK SWDIO (programming)
PA14	SWCLK	ST-LINK SWCLK (programming)
PC13	GPIO output	Onboard status LED
PH0/PH1	HSE crystal	25 MHz crystal (already on Black Pill)
3V3 / GND	Power	3.3 V devices: MAX9814, MCP4725
5V (VBUS)	Power	LM386 module VCC

4.2 Power Distribution

Critical rule: the Black Pill's onboard 3.3 V regulator can supply only ~300 mA — enough for the MAX9814 and MCP4725, but **not enough for the T-A7608 LTE module**, which can pull bursts of up to 2 A during transmission. The LTE module needs its own dedicated 5 V → 3.8 V supply (or a Li-ion battery with a buck regulator). All devices must share a common ground.



Decoupling: place a 470 µF electrolytic capacitor in parallel with a 100 nF ceramic capacitor across the LTE module's power input, as physically close to the module's VBAT pin as possible. This absorbs the inrush current during 4G transmit bursts and prevents brown-outs of the rest of the system.

4.3 Microphone Path (MAX9814 → STM32)

```
MAX9814 module                  Black Pill
     VDD ---------------------->  3V3
     GND ---------------------->  GND
     OUT ---------------------->  PA0  (ADC1_IN0)
     GAIN -- floating (default 60 dB) or tie to VDD (40 dB) / GND (50 dB)
     A/R  -- floating (default attack/release timing)
```

The MAX9814 OUT pin idles at approximately 1.25 V (its internal mid-rail bias). With ADC reference voltage of 3.3 V, the digital midpoint code is ≈ 96 (out of 255 in 8-bit mode). Firmware subtracts this offset to get a signed audio sample.

4.4 Speaker Path (STM32 → MCP4725 → LM386 → Headset)

```
MCP4725 module                  Black Pill
     VDD ---------------------->  3V3
     GND ---------------------->  GND
     SCL ---------------------->  PB6  (I2C1_SCL)
     SDA ---------------------->  PB7  (I2C1_SDA)
     A0  ---- tie to GND  (sets I2C address = 0x60)
     OUT ---------------------->  LM386 module IN

   LM386 module                    TRRS jack breakout
     VCC ----> 5V  (NOT 3.3V - LM386 needs >=4V)
     GND ----> GND
     IN  ----> from MCP4725 OUT
     +   ----> TRRS Tip   (Left audio)
     +   ----> TRRS Ring1 (Right audio)  -- tie L=R for mono
     GND ----> TRRS Sleeve (or Ring2, depending on wiring standard)
```

MCP4725 pull-ups: most breakouts already include 4.7 kΩ pull-ups on SCL and SDA. Do not add additional pull-ups. If your specific module lacks them, add a single pair of 4.7 kΩ resistors from each line to 3.3 V.

LM386 gain: if your LM386 module has a gain potentiometer, start with it fully down. The MCP4725 + LM386 combination is loud enough to clip badly at full gain.

4.5 LTE Module (T-A7608SA-H) Wiring

```
T-A7608 module                  Black Pill
     VBAT (3.3-4.3V) ---------->  dedicated 5V->3.8V buck or Li-ion
     GND ---------------------->  GND  (common with STM32 GND)
     TXD ---------------------->  PA10 (USART1_RX)
     RXD ---------------------->  PA9  (USART1_TX)
     PWRKEY ------------------->  pulse low >=1s to power on (button or GPIO+transistor)
     GND ---------------------->  GND  (a second ground wire - redundant at 2A)
```

UART logic levels: the T-A7608 UART operates at 1.8 V or 3.3 V depending on the breakout board. Verify before connecting. The STM32 outputs 3.3 V — if the LTE module requires 1.8 V, a level shifter is needed on the STM32 → LTE direction.

4.6 TRRS Headset Wiring (CTIA Standard)

A standard Android/iPhone TRRS headset uses the CTIA wiring standard:

Plug Section	Signal	Wire to
Tip	Left audio (output)	LM386 output
Ring 1	Right audio (output)	LM386 output (same line, mono)
Ring 2	Ground	Common GND
Sleeve	Microphone (input)	MAX9814 input pad (or leave floating to use onboard mic)

Note: if you use the onboard electret microphone on the MAX9814 breakout for the first prototype, leave the TRRS Sleeve disconnected. You only need the Tip + Ring 1 + Ring 2 (GND) connections for

headphone output. This is simpler for initial bring-up.

5. STM32CubeMX Configuration

Step-by-step procedure to configure the project in STM32CubeMX (or via the STM32 VS Code Extension). Follow this top-to-bottom for a fresh project.

5.1 Project Creation

- Launch STM32CubeMX → File → New Project.
- Search **STM32F401RCTx** → select your row → Start Project.
- You land on the Pinout & Configuration tab with a chip diagram.

5.2 Clock Source (RCC + SYS)

- System Core → RCC → High Speed Clock (HSE): **Crystal/Ceramic Resonator**.
- System Core → RCC → Low Speed Clock (LSE): **Disable**.
- System Core → SYS → Debug: **Serial Wire**. (Critical — without this, SWD programming fails after the first flash.)
- System Core → SYS → Timebase Source: **SysTick** (default).

5.3 Clock Configuration Tab — 84 MHz HCLK

Field	Value
Input Frequency	25 MHz
PLL Source Mux	HSE
PLLM	/25 (→ 1 MHz reference)
PLLN	×336 (→ 336 MHz VCO)
PLLP	/4 (→ 84 MHz SYSCLK)
PLLQ	/7 (don't care, no USB)
System Clock Mux	PLLCLK
AHB Prescaler	/1 (HCLK = 84 MHz)
APB1 Prescaler	/2 (PCLK1 = 42 MHz, TIM clk = 84 MHz)
APB2 Prescaler	/1 (PCLK2 = 84 MHz, TIM clk = 84 MHz)
Voltage Scaling	Scale 1
Flash Latency	2 WS (auto-set)

Shortcut: type 84 in the HCLK field and press Enter. CubeMX auto-solves the PLL chain.

5.4 TIM2 — 8 kHz ADC Trigger

- Timers → TIM2 → Clock Source: **Internal Clock**.
- Parameter Settings → Prescaler (PSC): **83**.
- Parameter Settings → Counter Period (ARR): **124**.
- Parameter Settings → auto-reload preload: **Enable**.
- Trigger Output → Master/Slave Mode: **Disable**.
- Trigger Output → Trigger Event Selection TRGO: **Update Event**.
- Math: 84 MHz / ((83+1) × (124+1)) = 8000 Hz exactly.

5.5 TIM3 — 8 kHz DAC Playback Tick

- Timers → TIM3 → Clock Source: **Internal Clock**.
- Parameter Settings → Prescaler: **83**, Counter Period: **124**.
- NVIC Settings → enable **TIM3 global interrupt**.

5.6 ADC1 — 8-bit, Timer-Triggered, DMA Circular

- Analog → ADC1 → check **IN0** (PA0 turns green).
- Parameter Settings → Clock Prescaler: **PCLK2 div 4** (21 MHz; ADC max is 36 MHz).
- Parameter Settings → Resolution: **8 bits**.
- Parameter Settings → Continuous Conversion Mode: **Disabled**.
- Parameter Settings → DMA Continuous Requests: **Enabled**.
- External Trigger Conversion Source: **Timer 2 Trigger Out event**.
- External Trigger Conversion Edge: **Rising edge**.
- Channel Sampling Time: 15 Cycles.

DMA Settings tab → Add → ADC1:

Field	Value
Stream	DMA2 Stream 0 (auto)
Direction	Peripheral To Memory
Mode	Circular
Priority	High
Use FIFO	Disabled (Direct mode)
Increment Address — Peripheral	Unchecked
Increment Address — Memory	Checked
Peripheral Data Width	Byte
Memory Data Width	Byte

NVIC Settings tab: enable *DMA2 stream0 global interrupt* and *ADC1 global interrupt*.

5.7 I2C1 — MCP4725 at 400 kHz Fast Mode

- Connectivity → I2C1 → Mode: **I2C**.
- Parameter Settings → I2C Speed Mode: **Fast Mode**.
- Parameter Settings → I2C Clock Speed (Hz): **400000**.
- Parameter Settings → Fast Mode Duty Cycle: 2.
- Pins auto-assign: PB6 = SCL, PB7 = SDA.
- (Optional) Add I2C1_TX DMA — Normal mode, Memory→Peripheral, byte width.

5.8 USART1 — LTE Module (115200 baud)

- Connectivity → USART1 → Mode: **Asynchronous**.
- Parameter Settings: Baud Rate **115200**, Word Length 8 bits, Parity None, Stop Bits 1.
- Hardware Flow Control: None (enable RTS/CTS only if your LTE module wiring uses them).
- NVIC Settings → enable **USART1 global interrupt**.
- Pins auto-assign: PA9 = TX, PA10 = RX.

5.9 USART2 — Optional Debug Console

- Connectivity → USART2 → Mode: **Asynchronous**, baud 115200, 8N1.
- NVIC: leave disabled (use blocking HAL_UART_Transmit for printf-style logging).
- Pins auto-assign: PA2 = TX, PA3 = RX. Wire PA2 to a USB-TTL adapter for live debug logs.

5.10 GPIO — Status LED on PC13

- On the chip diagram, click **PC13** → select **GPIO_Output**.
- GPIO config: output level High (LED is active-low on Black Pill), Push-Pull, Low speed, no pull.
- User Label: **LED**.

5.11 NVIC Priority Table

Interrupt	Preempt Priority	Reason
DMA2 Stream0 (ADC)	0 (highest)	Audio capture must never be starved
I2C1 EV / I2C1 ER	1	Audio playback timing
ADC1	2	Conversion complete signaling
TIM3	2	Playback tick
USART1	3	LTE bytes — least time-critical

5.12 Project Manager Settings

- Project Name: **SecureVoice**.
- Toolchain / IDE: **CMake** (for VS Code) or **STM32CubeIDE**.
- Linker Settings → Minimum Heap Size: **0x800**, Minimum Stack Size: **0x800**.
- Code Generator → tick **Generate peripheral initialization as a pair of '.c/.h' files per peripheral**.
- Click **GENERATE CODE**.

6. Software Architecture

6.1 Module Breakdown

ADC Driver — Continuously samples PA0 at 8 kHz via TIM2-triggered ADC + DMA into a circular ping-pong buffer. Generates HalfCplt and FullCplt callbacks signaling buffer halves are ready for processing.

DAC Driver — TIM3 ISR fires at 8 kHz and pushes the next playback sample to the MCP4725 over I²C using interrupt-driven transfers (HAL_I2C_Master_Transmit_IT).

Crypto Module — RSA-512 (PKCS#1 v1.5) for the initial session-key exchange. AES-128-CTR for bulk audio encryption using the exchanged key. RSA-only audio is not real-time feasible at 8 kHz on the F401.

LTE Driver — AT-command state machine over USART1. Brings up GPRS, opens a TCP connection to the relay server, performs REG/CALL/ANS handshake, then enters streaming mode.

Audio Buffer Manager — Coordinates the four producer/consumer flows: ADC→encrypt→TX, RX→decrypt→DAC. Uses ping-pong buffers to decouple sample timing from variable-latency processing.

Call State Machine — Tracks IDLE → REGISTERED → CALLING/RINGING → TALKING → IDLE. Drives the status LED to indicate state to the user.

6.2 Data Flow per Audio Frame

```
[TX path]  Mic --> MAX9814 --> ADC@8kHz --> ping-pong half --> AES encrypt
              --> UART TX --> T-A7608 --> TCP --> Relay --> peer T-A7608
              --> peer UART RX --> peer AES decrypt --> peer ping-pong half
              --> MCP4725 @ 8kHz --> LM386 --> Speaker

   [RX path]  Symmetric, runs in parallel on the same hardware.
```

6.3 Hybrid Encryption Rationale

Pure RSA-512 on every audio block cannot keep up with 8 kHz audio on a Cortex-M4 at 84 MHz. Decryption alone takes 40–100 ms per 53-byte block; 8 kHz audio requires processing roughly 150 blocks per second. The math simply does not work out.

Solution: use RSA only for what it is designed for — exchanging a small secret key. At session start, Device A generates a random 16-byte AES-128 key, encrypts it with Device B's RSA public key, and sends it. Device B decrypts the key with its RSA private key. From then on, both sides use AES-128-CTR for the bulk audio stream — sub-millisecond per block on the M4.

This is exactly how TLS, Signal, and every other production secure-comms system works ("hybrid encryption"). RSA = key exchange. AES = data.

6.4 Buffer Sizing

Parameter	Value	Notes
Sample rate	8 kHz	8000 samples/sec mono, voice-grade
Sample width	8 bits	unsigned, mid-bias 0x60 due to MAX9814
Audio frame size	128 samples	16 ms per frame — standard voice frame
Ping-pong total	256 bytes	two halves of 128 each
AES block size	16 bytes	AES-128 native block
RSA modulus	64 bytes	RSA-512 = 64 bytes ciphertext
Frames per second	~62.5	8000 / 128

Parameter	Value	Notes
LTE bandwidth	~10 KB/s	8 kHz × 1 byte + framing overhead

7. Relay Server (Laptop + ngrok)

The relay forwards encrypted bytes between the two devices. It runs on the developer's laptop during testing/demo and is exposed to the public internet via ngrok's free TCP tunnel.

7.1 Protocol Specification

All control messages are line-based ASCII terminated by '\n'. After the GO message, the connection switches to raw byte forwarding mode.

Direction	Message	Meaning
Client → Server	REG <phone>	Register this connection with a phone number
Server → Client	OK	Registration accepted
Server → Client	ERR <reason>	Command failed
Client → Server	CALL <peer_phone>	Initiate a call to peer
Server → Client	RING	Your CALL was accepted, peer is ringing
Server → Client	INC <caller>	(unsolicited) Someone is calling you
Client → Server	ANS	Pick up an incoming call
Server → Client	GO	Both ends now in audio-streaming mode
Client → Server	HUP	End the current call
Server → Client	PEER_HUP	Peer ended the call
Server → Client	PEER_GONE	Peer disconnected unexpectedly

7.2 Server Code (Python 3, asyncio)

Save as **relay.py** on the laptop. Run with python relay.py. Listens on TCP port 5555. No external dependencies.

```
import asyncio

CLIENTS = {}   # phone_number -> Session

class Session:
    def __init__(self, r, w):
        self.r = r
        self.w = w
        self.phone = None
        self.peer = None
        self.state = "IDLE"
        self.kick = asyncio.Event()

    async def send(self, line):
        self.w.write((line + "\n").encode())
        await self.w.drain()

async def handle_command(s, line):
    parts = line.split(maxsplit=1)
    if not parts: return
    verb = parts[0].upper()
    arg  = parts[1] if len(parts) > 1 else None

    if verb == "REG" and s.state == "IDLE":
        if not arg:           return await s.send("ERR no_phone")
        if arg in CLIENTS:    return await s.send("ERR taken")
        s.phone = arg; s.state = "REGD"; CLIENTS[arg] = s
        await s.send("OK")
        print(f"+ registered {arg}")
```

```
        elif verb == "CALL" and s.state == "REGD":
            peer = CLIENTS.get(arg)
            if not peer or peer.state != "REGD":
                return await s.send("ERR unavail")
            s.peer = peer; peer.peer = s
            s.state = "CALLING"; peer.state = "RINGING"
            await s.send("RING")
            await peer.send(f"INC {s.phone}")
            peer.kick.set()

        elif verb == "ANS" and s.state == "RINGING":
            s.state = "TALK"; s.peer.state = "TALK"
            await s.send("GO"); await s.peer.send("GO")
            s.peer.kick.set()

        elif verb == "HUP":
            if s.peer:
                await s.peer.send("PEER_HUP")
                s.peer.peer = None; s.peer.state = "REGD"; s.peer.kick.set()
            s.peer = None; s.state = "REGD"
            await s.send("OK")
        else:
            await s.send(f"ERR {verb}_in_{s.state}")

    async def setup_phase(s):
        while s.state not in ("TALK", "GONE"):
            rt = asyncio.create_task(s.r.readline())
            kt = asyncio.create_task(s.kick.wait())
            done, pending = await asyncio.wait([rt, kt],
                return_when=asyncio.FIRST_COMPLETED)
            for p in pending: p.cancel()
            if kt in done:
                s.kick.clear(); continue
            line = rt.result()
            if not line:
                s.state = "GONE"; return
            await handle_command(s, line.decode(errors="replace").strip())

    async def talk_phase(s):
        while s.state == "TALK" and s.peer and s.peer.state == "TALK":
            data = await s.r.read(256)
            if not data: break
            try:
                s.peer.w.write(data)
                await s.peer.w.drain()
            except (ConnectionResetError, BrokenPipeError):
                break

    async def handle(r, w):
        s = Session(r, w)
        print(f"+ conn {w.get_extra_info('peername')}")
        try:
            await setup_phase(s)
            if s.state == "TALK":
                await talk_phase(s)
        finally:
            if s.phone in CLIENTS: del CLIENTS[s.phone]
            if s.peer:
                try: s.peer.w.write(b"PEER_GONE\n"); await s.peer.w.drain()
                except: pass
                s.peer.peer = None; s.peer.state = "REGD"; s.peer.kick.set()
            try: w.close(); await w.wait_closed()
            except: pass

    async def main():
        srv = await asyncio.start_server(handle, "0.0.0.0", 5555)
        print(f"relay listening on {srv.sockets[0].getsockname()}")
        async with srv:
            await srv.serve_forever()

    if __name__ == "__main__":
```

```
try: asyncio.run(main())
except KeyboardInterrupt: print("\nbye")
```

7.3 Exposing the Relay via ngrok

- Sign up at ngrok.com (free tier).
- Download the Windows zip → unzip → single ngrok.exe.
- Add your auth-token: ngrok config add-authtoken <your_token>
- Start the tunnel (separate terminal, with relay.py also running): ngrok tcp 5555
- ngrok prints a public address such as **2.tcp.ngrok.io:14732**. This is the host:port the LTE devices will connect to.
- **Note:** the free tier assigns a new address each restart. Leave the tunnel running from demo morning.

7.4 Local Testing (no STM32 needed)

The relay can be fully validated before any embedded code exists. Open three terminals on the laptop:

- Terminal 1: python relay.py
- Terminal 2: telnet localhost 5555 → type REG 01000000001 → expect OK.
- Terminal 3: telnet localhost 5555 → type REG 01000000002 → expect OK.
- Terminal 2: CALL 01000000002 → expect RING; Terminal 3 should see INC 01000000001.
- Terminal 3: ANS → both terminals see GO.
- Now anything typed in either terminal appears in the other — bytes are forwarded transparently.

8. Incremental Bring-up Plan (Milestones)

Build and verify the system in slices. Do **not** wire all subsystems and then attempt to debug the whole thing at once — the project will not converge that way. Each milestone is independently testable.

M1: LED Blink

What: External LED on PA1 with 330 Ω resistor. Verifies the toolchain end-to-end (driver, IDE, ST-Link, board, GPIO config, clock tree).

Success criteria: LED blinks at 1 Hz.

M2: ADC Visualizer

What: MAX9814 → PA0 + USB-TTL adapter on USART2. Firmware streams ADC sample min/max to the PC terminal.

Success criteria: Numbers on terminal change visibly when whistling/speaking into the mic.

M3: Local Audio Loopback

What: Add MCP4725 + LM386 + TRRS jack. Mic samples are written directly to the DAC at 8 kHz with no encryption and no LTE.

Success criteria: Hear yourself in the headset with ~32 ms delay. Proves the entire analog/digital audio chain.

M4: LTE Module AT Sanity

What: Wire T-A7608 to USB-TTL adapter directly (not yet to STM32). Manually issue AT+CIPSTART to a public TCP echo server.

Success criteria: Module powers on, registers on the network, opens TCP, echoes data.

M5: STM32-Driven TCP Echo

What: Connect T-A7608 to STM32 USART1. Firmware automates the AT sequence and echoes 100 bytes/s of dummy data through the relay.

Success criteria: Bytes loop back via the relay; first end-to-end network proof.

M6: Two-Node Plain Audio over LTE

What: Both devices fully assembled. Audio flows mic→ADC→LTE→relay→LTE→DAC→speaker, no encryption yet.

Success criteria: First two-way voice between the devices, even if quality is rough.

M7: Add RSA + AES Hybrid Encryption

What: Drop in mbedTLS. Pre-share RSA keypairs, exchange AES session key at call setup, encrypt audio frames with AES-CTR.

Success criteria: All voice traffic on the LTE link is unintelligible to a packet sniffer; decoded perfectly at the peer.

M8: Phone-Number Call Setup + Polish

What: Add the REG/CALL/ANS state machine, status LED, push-button to initiate/answer calls.

Success criteria: User experience matches a real phone call.

9. Firmware Code Skeleton

Reference snippets to drop into the CubeMX-generated **main.c** at the marked USER CODE regions. These are starting points — Milestones 2, 3, and 5 use progressively larger subsets.

9.1 Variables and Buffers

```
/* USER CODE BEGIN PD */
#define AUDIO_BUFFER_SIZE   256        /* total ping-pong, samples */
#define HALF_BUFFER_SIZE    (AUDIO_BUFFER_SIZE / 2)
#define LTE_RX_BUFFER_SIZE  512
#define LTE_TX_BUFFER_SIZE  512
#define MCP4725_ADDR        (0x60 << 1)
/* USER CODE END PD */

/* USER CODE BEGIN PV */
uint8_t  adc_buffer[AUDIO_BUFFER_SIZE];   /* mic capture (DMA target) */
uint8_t  dac_buffer[AUDIO_BUFFER_SIZE];   /* speaker source           */

volatile uint8_t adc_half_ready = 0;
volatile uint8_t adc_full_ready = 0;
volatile uint16_t dac_play_idx  = 0;

uint8_t lte_rx_buffer[LTE_RX_BUFFER_SIZE];
uint8_t lte_tx_buffer[LTE_TX_BUFFER_SIZE];
volatile uint8_t lte_rx_ready = 0;
/* USER CODE END PV */
```

9.2 Peripheral Start-up

```
/* USER CODE BEGIN 2 */
HAL_UART_Receive_IT(&huart1, lte_rx_buffer, LTE_RX_BUFFER_SIZE);
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, AUDIO_BUFFER_SIZE);
HAL_TIM_Base_Start(&htim2);
HAL_TIM_Base_Start_IT(&htim3);
/* USER CODE END 2 */
```

9.3 Callbacks

```
/* USER CODE BEGIN 4 */
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *h) {
    if (h->Instance == ADC1) adc_half_ready = 1;
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *h) {
    if (h->Instance == ADC1) adc_full_ready = 1;
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h) {
    if (h->Instance == USART1) {
        lte_rx_ready = 1;
        HAL_UART_Receive_IT(&huart1, lte_rx_buffer, LTE_RX_BUFFER_SIZE);
    }
}

/* TIM3 @ 8 kHz: push next sample to MCP4725 over I2C */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM3) {
        uint16_t v12 = ((uint16_t)dac_buffer[dac_play_idx]) << 4;
        uint8_t pkt[2] = {
            (uint8_t)((v12 >> 8) & 0x0F),
            (uint8_t)(v12 & 0xFF)
        };
        HAL_I2C_Master_Transmit_IT(&hi2c1, MCP4725_ADDR, pkt, 2);
        dac_play_idx = (dac_play_idx + 1) % AUDIO_BUFFER_SIZE;
    }
}
/* USER CODE END 4 */
```

9.4 Main Loop

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /* TX: encrypt completed half-buffer & ship to LTE */
    if (adc_half_ready) {
        adc_half_ready = 0;
        Encrypt_Frame(&adc_buffer[0], HALF_BUFFER_SIZE, lte_tx_buffer);
        LTE_Send(lte_tx_buffer, /*ciphertext_len*/);
    }
    if (adc_full_ready) {
        adc_full_ready = 0;
        Encrypt_Frame(&adc_buffer[HALF_BUFFER_SIZE], HALF_BUFFER_SIZE, lte_tx_buffer);
        LTE_Send(lte_tx_buffer, /*ciphertext_len*/);
    }

    /* RX: decrypt incoming frame & write into the playback half */
    if (lte_rx_ready) {
        lte_rx_ready = 0;
        Decrypt_Frame(lte_rx_buffer, /*len*/, dac_buffer + /*write_offset*/);
    }

    HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
}
/* USER CODE END WHILE */
```

9.5 LTE AT-Command Bring-up Sketch

```
static void LTE_Init(void) {
    HAL_Delay(3000);  /* boot delay */

    LTE_AT("AT",                                      1000);
    LTE_AT("ATE0",                                    1000);
    LTE_AT("AT+CPIN?",                                2000);
    LTE_AT("AT+CSQ",                                  1000);
    LTE_AT("AT+CGATT=1",                             10000);
    LTE_AT("AT+CGDCONT=1,\"IP\",\"<YOUR_APN>\"",  2000);
    LTE_AT("AT+CGACT=1,1",                           15000);
    LTE_AT("AT+CIPMUX=0",                             2000);
    LTE_AT("AT+CIPSTART=\"TCP\",\"<NGROK_HOST>\","
           "<NGROK_PORT>",                           20000);
    /* Expect "CONNECT OK". Then: */
    LTE_Write("REG <YOUR_PHONE>\r\n");
    /* Wait for "OK" → ready for CALL / INC */
}
```

10. Development Timeline

Week	Phase	Deliverable
9	Planning, procurement, environment setup	Toolchain ready; all components in hand; LED blink working (M1)
10	Hardware assembly & ADC capture	MAX9814 wired; ADC visualizer working (M2); MCP4725+LM386 wired
11	Audio pipeline & local loopback	Mic→ADC→DAC→speaker loopback working (M3)
12	LTE integration	AT-command bring-up (M4); STM32-driven TCP echo (M5); plain audio over
13	Encryption layer	mbedTLS integrated; RSA+AES hybrid encryption working (M7)
14	Polish & call control	Phone-number call setup, status LED, push buttons (M8)
15	Final testing & demonstration	Reliability testing, latency measurement, demo prep, video backup

10.1 Latency Budget

Stage	Estimated Latency
ADC fill (one half-buffer at 8 kHz × 128 samples)	16 ms
AES-128-CTR encrypt of 128 bytes	< 1 ms
UART → LTE module → 4G transmit	50–150 ms (network)
Relay forwarding	< 5 ms
4G receive → UART → STM32 RX	50–150 ms (network)
AES decrypt of 128 bytes	< 1 ms
DAC playback fill	16 ms
Estimated one-way mouth-to-ear	~150–350 ms

The cellular network dominates the latency budget. The encryption objective (under 200 ms) refers to the on-device processing path, which is comfortably achieved by the hybrid scheme.

11. Risk Register & Mitigations

Risk	Cause	Mitigation
LTE module brown-out during transmit	A7608 pulls up to 2 A in bursts; weak supply causes the module to reset.	Use a dedicated 5 V/2 A regulator for the LTE module. Add 470
ADC offset / clipping	MAX9814 output sits at ~1.25 V; ADC reads 96/255 at silence. Naive playback sounds garbled.	Subtract 96 from sample before playback; use signed audio i
Pure RSA cannot keep up with audio	RSA-512 decrypt = 40-100 ms per block; audio needs 150 blocks/sec.	Use hybrid: RSA for key exchange only, AES-128-CTR for bulk
CGNAT prevents direct connection	Two LTE modules cannot directly TCP-connect to each other.	Relay server (laptop + ngrok). Architecturally identical to all rea
ngrok address changes on restart	Free tier reassigns the public address each time the tunnel restarts.	Start the tunnel once in the morning and leave it running. Ha
Demo failure on the day	Live cellular and audio demos are notoriously fragile.	Record a working video demo as backup. Have the milestone t
RSA-512 is cryptographically weak	RSA-512 is broken in practice; not suitable for real-world use.	Acknowledge this explicitly in the report. The objective is to dem
Data plan exhaustion	Continuous voice consumes ~36 MB/hour of data per node.	Use a SIM with at least 1 GB/month data; estimate 10-30 hours

12. Glossary

Term	Definition
ADC	Analog-to-Digital Converter. Samples analog voltage into digital codes.
AES-128-CTR	Advanced Encryption Standard with 128-bit key in Counter mode. Symmetric stream cipher; fast on Co
APN	Access Point Name. The cellular gateway identifier required to attach a data session.
CGNAT	Carrier-Grade NAT. Network address translation done by mobile carriers, hiding all subscribers behind
DAC	Digital-to-Analog Converter. Reconstructs an analog signal from digital samples.
DMA	Direct Memory Access. Lets peripherals move data to/from RAM without CPU involvement.
Full-duplex	Simultaneous two-way communication, like a real phone call (vs. half-duplex like a walkie-talkie).
HAL	Hardware Abstraction Layer. ST's C library that wraps STM32 peripheral registers in friendlier function
HCLK	AHB system bus clock; the main CPU clock on STM32.
Hybrid encryption	Use asymmetric crypto (RSA) to exchange a symmetric key, then symmetric crypto (AES) for bulk data
I²C	Two-wire serial bus (SDA + SCL) used to talk to the MCP4725 DAC.
ngrok	Free tunneling service that exposes a port on your laptop to the public internet via a forwarded address
RSA-512	Asymmetric (public-key) encryption with 512-bit modulus. Weak by modern standards but light enough
SWD	Serial Wire Debug. Two-wire programming/debug interface (SWDIO + SWCLK) used by the ST-Link.
TCP	Transmission Control Protocol. Reliable, ordered byte-stream over IP — used for the relay link.
TRGO	Trigger Output. STM32 timer feature that lets one peripheral (e.g., TIM2) trigger another (e.g., ADC).
TRRS	Tip-Ring-Ring-Sleeve. Four-conductor 3.5 mm jack standard, used by mic+headphone headsets.
VoLTE	Voice over LTE. Carrier-managed voice call over the LTE data network — bypassed in this project.

End of document.